
Summary of Optimization Methods in `scipy.optimize.minimize`

Conjugate Gradient (CG)

Type: First-order, gradient-based

Uses: Function values + gradients

- Very low memory requirements, making it suitable for problems where storing a Hessian is impractical.
- Works well when the objective is smooth and gradients are easy to compute.
- More efficient than basic gradient descent because it constructs conjugate search directions.
- Performs well on medium-scale problems where Hessian computation is too expensive.
- **Limitations:** May struggle on functions with curved, narrow valleys (such as Rosenbrock), and can exhibit zig-zagging if poorly conditioned.

Typical problem size:

Mid-size: approximately 50 to a few hundred variables.

BFGS (Quasi-Newton)

Type: First-order, Hessian approximation method

Uses: Function values + gradients

- Builds and updates an approximation of the Hessian (curvature matrix) without computing it directly.
- Fast and stable convergence on smooth optimization problems.
- Often a strong default choice for unconstrained optimization.
- Performs particularly well on objectives with significant curvature, such as the Rosenbrock function.
- **Limitations:** Memory usage grows quadratically with the number of variables, limiting scalability.

Typical problem size:

Moderate: 2 to roughly 500 variables.

Summary of Optimization Methods in `scipy.optimize.minimize`

Newton–CG

Type: Second-order method using Hessian–vector products

Uses: Gradient + Hessian or Hessian–vector products

- Achieves fast, often superlinear convergence near the optimum.
- More memory-efficient than full Newton’s method because it does not store the entire Hessian.
- Particularly useful when Hessian-vector products can be computed efficiently (common in machine learning).
- Scales well to larger problems where curvature information is important but full Hessian formation is too expensive.
- **Limitations:** Requires accurate Hessian or Hessian-vector products; may struggle with non-convex regions.

Typical problem size:

Mid-size to large: approximately 100 to 10,000 variables.

Nelder–Mead

Type: Derivative-free simplex search

Uses: Function values only (no gradients)

- Useful for optimization when gradients do not exist, are expensive to compute, or the objective function is noisy.
- Moves a simplex (a geometric shape) through the space via reflection, expansion, contraction, and shrink steps.
- Very robust on small, low-dimensional problems with irregular surfaces.
- **Limitations:** Slow, does not guarantee convergence to a true optimum, and does not scale to high dimensions.

Typical problem size:

Small: 1 to 5 variables.